Code Review Exercise: Online Shopping Cart

```java
package generics.review;

import java.util.*;

class Product {
    private final long id;
    private final String name;
    private long price; // in cents
    private int stock;

    public Product(long id, String name, long price, int stock) {
        if (price < 0 || stock < 0) {
            throw new IllegalArgumentException("Price and stock must be non-negative");
        }
        this.id = id;
        this.name = Objects.requireNonNull(name);
        this.price = price;
        this.stock = stock;
    }

    public long getId() { return id; }
    public String getName() { return name; }
    public long getPrice() { return price; }
    public int getStock() { return stock; }

    public void setPrice(long price) {
        if (price < 0) {
            throw new IllegalArgumentException("Price must be non-negative");
        }
        this.price = price;
    }

    public boolean isInStock() {
        if (stock > 0) {
            return true;
        } else {
            return false;
        }
    }

    public void decreaseStock(int quantity) {
        if (quantity > stock) {
            throw new IllegalStateException("Not enough stock for " + name);
        }
        stock -= quantity;
    }

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) {
            return false;
        }
        Product product = (Product) o;
        return id == product.id;
    }

    @Override
    public int hashCode() {
        return Objects.hashCode(id);
    }
}

class CartItem {
    private final Product product;
    private int quantity;

    public CartItem(Product product, int quantity) {
        this.product = Objects.requireNonNull(product);
        this.quantity = quantity;
    }

    public Product getProduct() { return product; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }

    public long getSubtotal() { return product.getPrice() * quantity; }
}

class Order {
    private final String orderId = UUID.randomUUID().toString();
    private final List<CartItem> items;
    private final long total;

    public Order(Collection<CartItem> items, long total) {
        this.items = new ArrayList<>(items);
        this.total = total;
    }

    public String getOrderId() { return orderId; }
    public List<CartItem> getItems() { return items; }
    public long getTotal() { return total; }
}

```

Code Review Exercise: Online Shopping Cart

```java
class ProductCatalog {
    private final Map<Long, Product> products = new HashMap<>();

    public boolean addProduct(Product product) {
        if (products.containsKey(product.getId())) {
            return false;
        }
        products.put(product.getId(), product);
        return true;
    }

    public Product findProduct(long id) {
        return products.get(id);
    }

    public Collection<Product> getAllProducts() {
        return products.values();
    }

    public void removeOutOfStock() {
        for (Product product : products.values()) {
            if (!product.isInStock()) {
                products.remove(product.getId());
            }
        }
    }
}

class ShoppingCart {
    private final ProductCatalog catalog;
    private final Map<Long, CartItem> items = new LinkedHashMap<>();

    public ShoppingCart(ProductCatalog catalog) {
        this.catalog = Objects.requireNonNull(catalog);
    }

    public void addProduct(int productId, int quantity) {
        Product product = catalog.findProduct(productId);
        if (product == null) {
            throw new IllegalArgumentException("Unknown product: " + productId);
        }

        CartItem existing = items.get(product.getId());
        if (existing != null) {
            existing.setQuantity(existing.getQuantity() + quantity);
        } else {
            if (quantity > product.getStock()) {
                throw new IllegalArgumentException("Not enough stock for " + product.getName());
            }
            items.put(product.getId(), new CartItem(product, quantity));
        }
    }

    public void removeProduct(int productId) {
        items.remove(productId);
    }

    public long calculateTotal() {
        long total = 0;
        for (CartItem item : items.values()) {
            total += item.getSubtotal();
        }
        if (total < 5000) {
            total += 499;
        }
        return total;
    }

    public Order checkout() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Cart is empty");
        }

        for (CartItem item : items.values()) {
            item.getProduct().decreaseStock(item.getQuantity());
        }

        Order order = new Order(items.values(), calculateTotal());
        items.clear();
        return order;
    }

    public List<CartItem> getItems() {
        return new ArrayList<>(items.values());
    }
}
```

SHOP-142 — Product catalog, shopping cart & checkout

Implement the core of the online shop: a product catalog, a shopping cart and order checkout.

- Customers add products by id and quantity, can add the same product again later, and can remove a product from the cart.
- Shipping costs 4.99; orders of 50.00 or more ship for free. All prices are in cents.
- Checkout turns the cart into an order, reduces stock and empties the cart.
- Admins can add products to the catalog (product ids are unique) and remove sold-out products.